

Projet PC3R

RAMANDIMBISON Fy, 21522722

ATSE Yapi, 3803860

CinéSocial — Plan de la Partie Serveur

1. Approche utilisée

Le serveur de CinéSocial est écrit en Go à l'aide du package standard `net/http`, sans framework web. L'approche adoptée est celle d'une API REST orientée ressources, où chaque endpoint correspond à une ressource métier (utilisateurs, films, commentaires, notes, watchlist). Le client React communique exclusivement avec le serveur via des requêtes AJAX asynchrones, et le serveur répond systématiquement avec des données encodées en JSON. L'application suit une architecture monopage (SPA) côté client, le serveur ne générant aucune vue HTML. L'authentification est gérée par des tokens JWT transmis dans le header `Authorization` de chaque requête protégée.

2. Description des composants

handlers/auth.go — Authentification

Rôle : Gérer l'authentification des utilisateurs et la génération de tokens JWT.

Méthode	Route	Description
POST	/login	Vérifie email et mot de passe, génère et retourne un token JWT

handlers/utilisateurs.go — Utilisateurs

Rôle : Gérer la création et la consultation des profils utilisateurs.

Méthode	Route	Description
POST	/utilisateurs	Crée un nouvel utilisateur avec mot de passe hashé (bcrypt)
GET	/get/utilisateurs	Retourne la liste des utilisateurs
DELETE	/utilisateurs?id={id}	Supprime un utilisateur par son identifiant

handlers/films.go — Films

Rôle : Servir de proxy entre le client et l'API externe TMDB. Le serveur contacte TMDB à la demande et retransmet les résultats au client en JSON.

Méthode	Route	Description
GET	/films/tendance	Retourne les films tendance du jour via TMDB
GET	/films/recherche?q={titre}	Recherche des films par titre via TMDB

handlers/commentaires.go — Commentaires

Rôle : Gérer les commentaires laissés par les utilisateurs sur les films.

Méthode	Route	Description
POST	/commentaires	Crée un commentaire sur un film (protégé JWT)
GET	/commentaires?tmdb_id={id}	Retourne tous les commentaires d'un film

handlers/notes.go — Notes

Rôle : Gérer les notes attribuées par les utilisateurs aux films.

Méthode	Route	Description
POST	/notes	Ajoute ou modifie une note (protégé JWT)
GET	/notes?tmdb_id={id}	Retourne la note moyenne d'un film et la note de l'utilisateur

handlers/watchlist.go — Watchlist

Rôle : Gérer la liste personnelle de films à voir de chaque utilisateur.

Méthode	Route	Description
POST	/watchlist	Ajoute un film à la watchlist
GET	/watchlist	Retourne la watchlist de l'utilisateur connecté
DELETE	/watchlist?tmdb_id={id}	Supprime un film de la watchlist (protégé JWT)

middleware/auth.go — Middleware JWT

Rôle : Intercepter les requêtes vers les routes protégées et vérifier la validité du token JWT avant de laisser passer la requête au handler.

3. Extraits de code Connexion BDD — db/db.go

```
func Connect() (*sql.DB, error) {
    connStr := fmt.Sprintf(
        "host=%s port=%s user=%s password=%s dbname=%s sslmode=disable",
        os.Getenv("DB_HOST"),
        os.Getenv("DB_PORT"),
        os.Getenv("DB_USER"),
        os.Getenv("DB_PASSWORD"),
        os.Getenv("DB_NAME"),
    )
    return sql.Open("postgres", connStr)
}
```

Authentification — handlers/auth.go

```
func Login(db *sql.DB, w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodPost {
        http.Error(w, "Méthode non autorisée", http.StatusMethodNotAllowed)
        return
    }
    // 1. Lire le body
    var u models.Utilisateur
    err := json.NewDecoder(r.Body).Decode(&u)
    if err != nil {
        http.Error(w, "JSON invalide", http.StatusBadRequest)
        return
    }
    // 2. Chercher l'utilisateur en BDD par email
    var utilisateurBDD models.Utilisateur
    err = db.QueryRow(
        "SELECT id, nom_u, mot_de_passe FROM utilisateurs WHERE email = $1",
        u.Email,
    ).Scan(&utilisateurBDD.Id, &utilisateurBDD.NomUtilisateur, &utilisateurBDD.MotDePasse)
    if err != nil {
        http.Error(w, "Email ou mot de passe incorrect", http.StatusUnauthorized)
        return
    }
    // 3. Vérifier le mot de passe
    err = bcrypt.CompareHashAndPassword([]byte(utilisateurBDD.MotDePasse), []byte(u.MotDePasse))
    if err != nil {
        http.Error(w, "Email ou mot de passe incorrect", http.StatusUnauthorized)
    }
}
```

```

        return
    }
    // 4. Générer le token JWT
    token := jwt.NewWithClaims(jwt.SigningMethodHS256, jwt.MapClaims{
        "user_id": utilisateurBDD.Id,
        "nom_u": utilisateurBDD.NomUtilisateur,
        "exp":      time.Now().Add(24 * time.Hour).Unix(),
    })

    tokenString, err := token.SignedString([]byte(os.Getenv("JWT_SECRET")))
    if err != nil {
        fmt.Println("Erreur génération token :", err)
        http.Error(w, "Erreur serveur", http.StatusInternalServerError)
        return
    }

    // 5. Renvoyer le token
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(map[string]string{
        "token": tokenString,
    })
}

```

Films — handlers/films.go

```

func GetFilmsTendance(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodGet {
        http.Error(w, "Méthode non autorisée", http.StatusMethodNotAllowed)
        return
    }

    // 1. Appel à l'API TMDB
    url := fmt.Sprintf(
        "%strending/movie/day?api_key=%s&language=fr-FR",
        os.Getenv("TMDB_API_URL"), os.Getenv("TMDB_API_KEY"),
    )

    resp, err := http.Get(url)
    if err != nil {
        fmt.Println(err)
        http.Error(w, "Erreur appel TMDB", http.StatusInternalServerError)
        return
    }
    defer resp.Body.Close()

    // 2. Décoder la réponse TMDB
    var result map[string]interface{}
    json.NewDecoder(resp.Body).Decode(&result)
}

```

```
// 3. Renvoyer au client
w.Header().Set("Content-Type", "application/json")
json.NewEncoder(w).Encode(result)
}
```